JavaScript is disabled on your browser.

compact1, compact2, compact3

java.lang

Interface Iterable<T>

Type Parameters:

`T - the type of elements returned by the iterator`

All Known Subinterfaces:

BeanContext, BeanContextServices, BlockingDeque<E>, BlockingQueue<E>, Collection<E>, Deque<E>, DirectoryStream<T>, List<E>, NavigableSet<E>, Path, Queue<E>, SecureDirectoryStream<T>, Set<E>, SortedSet<E>, TransferQueue<E>

All Known Implementing Classes:

AbstractCollection, AbstractList, AbstractQueue, AbstractSequentialList, AbstractSet, ArrayBlockingQueue, ArrayDeque, ArrayList, AttributeList, BatchUpdateException, BeanContextServicesSupport, BeanContextSupport, ConcurrentHashMap.KeySetView, ConcurrentLinkedDeque, ConcurrentLinkedQueue, ConcurrentSkipListSet, CopyOnWriteArrayList, CopyOnWriteArraySet, DataTruncation, DelayQueue, EnumSet, HashSet, JobStateReasons, LinkedBlockingDeque, LinkedBlockingQueue, LinkedHashSet, LinkedList, LinkedTransferQueue, PriorityBlockingQueue, PriorityQueue, RoleList, RoleUnresolvedList, RowSetWarning, SerialException, ServiceLoader, SQLClientInfoException, SQLDataException, SQLException, SQLFeatureNotSupportedException, SQLIntegrityConstraintViolationException, SQLInvalidAuthorizationSpecException, SQLNonTransientConnectionException, SQLNonTransientException, SQLRecoverableException, SQLSyntaxErrorException, SQLTimeoutException, SQLTransactionRollbackException, SQLTransientConnectionException, SQLTransientException, SQLWarning, Stack, SyncFactoryException, SynchronousQueue, SyncProviderException, TreeSet, Vector

---

```
public interface Iterable<T>
```

Implementing this interface allows an object to be the target of the "for-each loop" statement. See **For-each Loop**

Since:

1.5

See *The Java™ Language Specification*:

14.14.2 The enhanced for statement

Method Summary

Abstract Methods | Default Methods

Modifier and Type	Method	Description
default void	**forEach**(**Consumer**<? super **T**> action)	Performs the given action for each element of the Iterable until all elements have been processed or the action throws an exception.
Iterator<**T**>	**iterator**()	Returns an iterator over elements of type T.
default **Spliterator**<**T**>	**spliterator**()	Creates a **Spliterator** over the elements described by this Iterable.

Method Detail

iterator

Iterator<T> iterator()

Returns an iterator over elements of type T.

Returns:

an Iterator.

forEach

default void forEach(Consumer<? super T> action)

Performs the given action for each element of the `Iterable` until all elements have been processed or the action throws an exception. Unless otherwise specified by the implementing class, actions are performed in the order of iteration (if an iteration order is specified). Exceptions thrown by the action are relayed to the caller.

Implementation Requirements:

The default implementation behaves as if:

```
for (T t : this)
    action.accept(t);
```

Parameters:

`action` - The action to be performed for each element

Throws:

`NullPointerException` - if the specified action is null

Since:

1.8

spliterator

```
default Spliterator<T> spliterator()
```

Creates a `Spliterator` over the elements described by this `Iterable`.

Implementation Requirements:

The default implementation creates an *early-binding* spliterator from the iterable's Iterator. The spliterator inherits the *fail-fast* properties of the iterable's iterator.

Implementation Note:

The default implementation should usually be overridden. The spliterator returned by the default implementation has poor splitting capabilities, is unsized, and does not report any spliterator characteristics. Implementing classes can nearly always provide a better implementation.

Returns:

a Spliterator over the elements described by this Iterable.

Since:

```
1.8
```

C

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.